Sensitivity workflow

Purpose

A sensitivity analysis shows which parameters change a chosen model result the most, so that calibration can concentrate on them. SWAT+ does not include a sensitivity analysis. It is run by an external tool that writes `calibration.cal`, runs SWAT+, reads a result from the output files, and repeats for many parameter sets. Each run applies the changes in `calibration.cal` at start-up and leaves the input files untouched, so one project folder serves every run.

SWAT+ Toolbox does this from a graphical interface. The steps below are what any tool or script has to do.

Steps

1. Choose the parameters to test and a range for each. The names are those on Parameter names. The `abs_min` and `abs_max` values in `cal_parms.cal` are physical limits; a narrower range around reasonable values usually gives more useful results.
2. Generate the samples: use a sampling design that matches the analysis method: Sobol (Saltelli sampling), FAST, Morris (elementary effects), Latin hypercube with one-at-a-time changes (LH-OAT), and so on. Each sample is one set of parameter values. Libraries such as SALib (Python) or `sensitivity` (R) provide both the sampling and the analysis.
3. For each sample, write `calibration.cal` with one row per parameter, run SWAT+ in the project folder, and compute the result of interest: total or peak flow at an outlet, a performance measure against observations (NSE, KGE, percent bias), a sediment or nutrient load.
4. Compute the indices: pass the sample matrix and the vector of results to the analysis method: first-order and total-order indices for Sobol and FAST, mean and spread of elementary effects for Morris.

The runs are independent, so they can be spread over several processor cores, each in a separate copy of the project folder. Sample sizes of several hundred to a few thousand runs are common.

Writing calibration.cal

A sample with three parameters, written as relative and absolute changes:

```
sensitivity sample 17
3
NAME      CHG_TYP      VAL    CONDS  LYR1  LYR2  YEAR1  YEAR2  DAY1  DAY2  OBJ_TOT
cn2       pctchg     -6.2     0      0     0     0      0      0     0     0
esco      absval      0.71    0      0     0     0      0      0     0     0
alpha     absval      0.048   0      0     0     0      0      0     0     0
```

Points to watch:

- `pctchg` scales each object's current value, so HRUs keep their differences. `absval` sets every object to the same value.
- Every parameter name must also be in `cal_parms.cal`, and its limits there still apply. A sample value outside `abs_min` to `abs_max` is cut to the limit, which flattens the response near the edges.
- A misspelt name is skipped without a message. Check a first run by comparing two extreme samples before starting the full set.
- Some parameter names have no effect in 62.0.1, or work only with restrictions; see the notes on Parameter change files and Hard calibration before including them.

Reading results

Daily channel output is in `channel_sd_day.txt` (or `.csv`), with flow out of each channel in the `flo_out` column (m^3/s). Switch on only the output the analysis needs in `print.prt`; writing every file for thousands of runs costs a lot of time and disk space. The output files are described in the Output Reference.

Related pages

- Parameter change files. Format of `cal_parms.cal` and `calibration.cal`, and the parameter names.
- Hard calibration. How each run applies the change list.

Sources

- The change list is read by `src/cal_parmchg_read.f90` and applied by `src/cal_conditions.f90`, both from `src/proc_cal.f90` before the simulation starts.